Problem and requirements

Project scope / shared organizational service desk

Problem

A failing payment application can generate multiple reports before support knows its reach. A queue ordered only by arrival can bury a widespread outage beneath routine requests. Separate channels also make it difficult to identify who acknowledged the incident and when.

Employee

Report an affected service, describe the symptoms, choose impact and urgency, follow progress, answer questions, verify a resolution and give feedback. Employees need access to their own history.

Technician

Review assigned work, acknowledge ownership, record investigations, ask for more information and supply a resolution. Workload and upcoming commitments must be visible.

Administrator

Manage the service catalog, users, categories, routing and SLA targets. Inspect individual incidents, audit events, workload, deadline violations and service-level trends.

Business rules

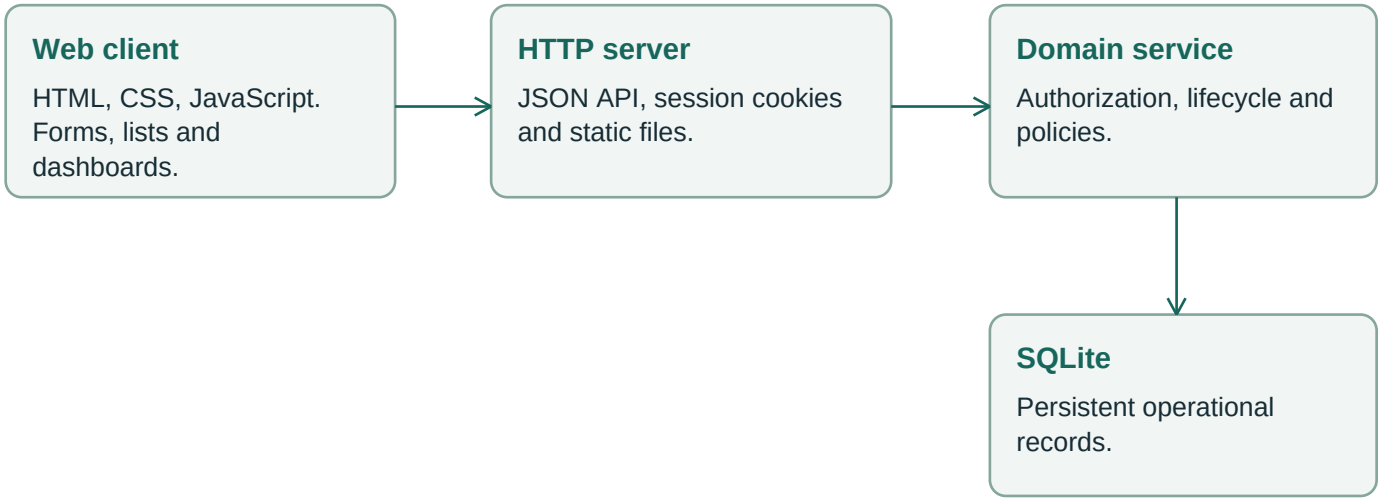
The server determines access from the authenticated user. Qualification, availability and active workload guide routing. Response and resolution have separate targets. Completion follows investigation and resolution, with reporter verification before closure.

Quality and operating assumptions

This implementation targets a single organization and a modest departmental queue. It needs durable records, safe simultaneous updates, keyboard-accessible forms, readable mobile layouts and a reproducible installation. In-app notices and simulated email support a demonstration without a mail provider.

Architecture and technology

Three layers / one deployable application



Language and backend

Java 17 supplies interfaces, records, cryptography and an HTTP server. Gson handles JSON and the SQLite JDBC driver handles persistence. Maven packages these dependencies with the browser resources in one runnable JAR. JUnit tests exercise rules independently of the interface.

Browser interface

Native controls and JavaScript let all three roles use the same browser entry point. The role-specific views share the same server-side rules. React would add a component ecosystem and a build toolchain.

Storage and trade-offs

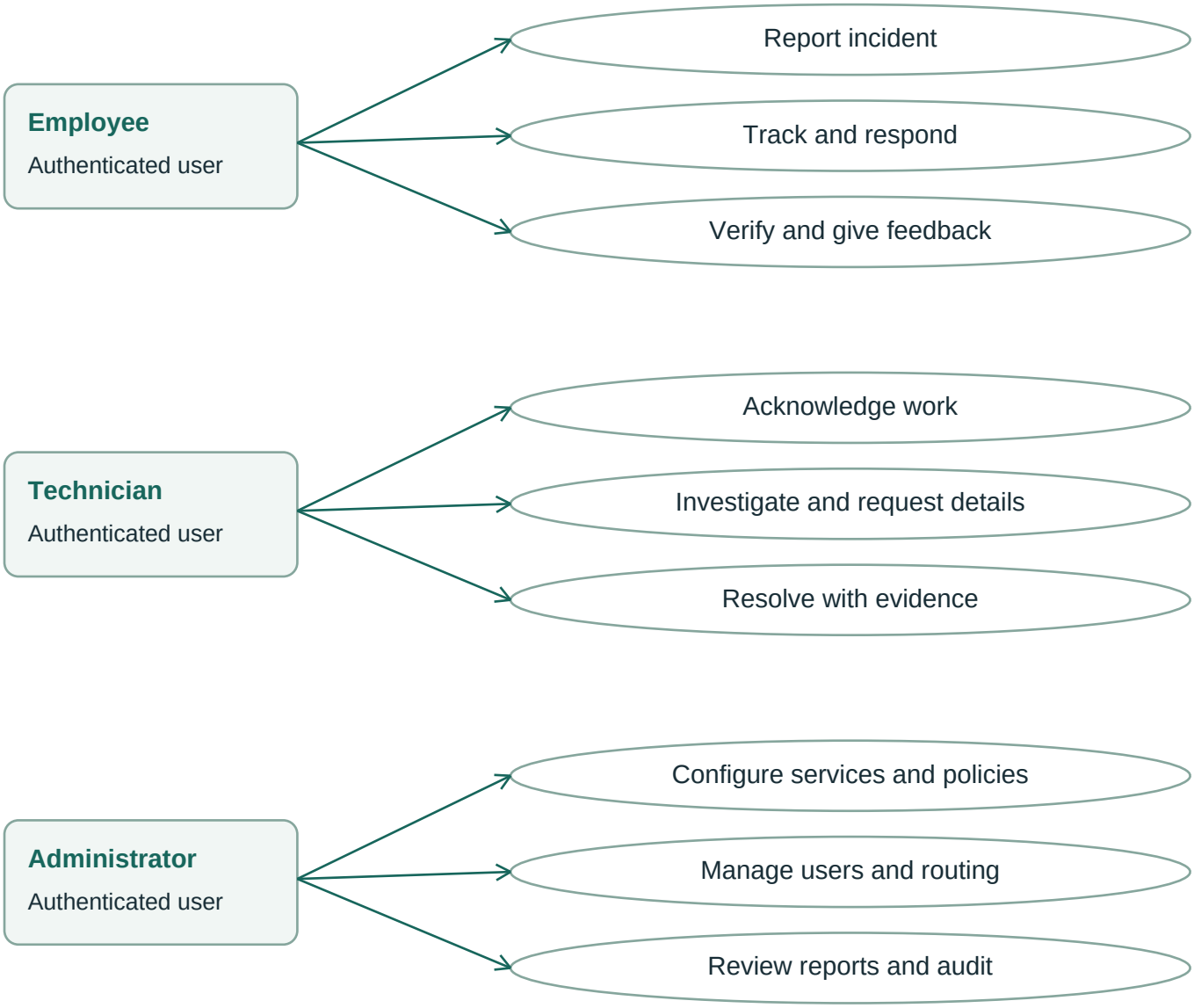
SQLite provides transactions and relational joins in one file. PostgreSQL would better support a large number of concurrent writers. A document store would require additional care to maintain relationships among users, services and incident events.

Growth path

Move storage to PostgreSQL, paginate and index high-volume queries, give SLA polling a dedicated worker, and deliver email through a retryable queue. Deploy behind HTTPS with a production server and integrate organizational identity.

Roles and use cases

Use-case diagram / actions by primary actor

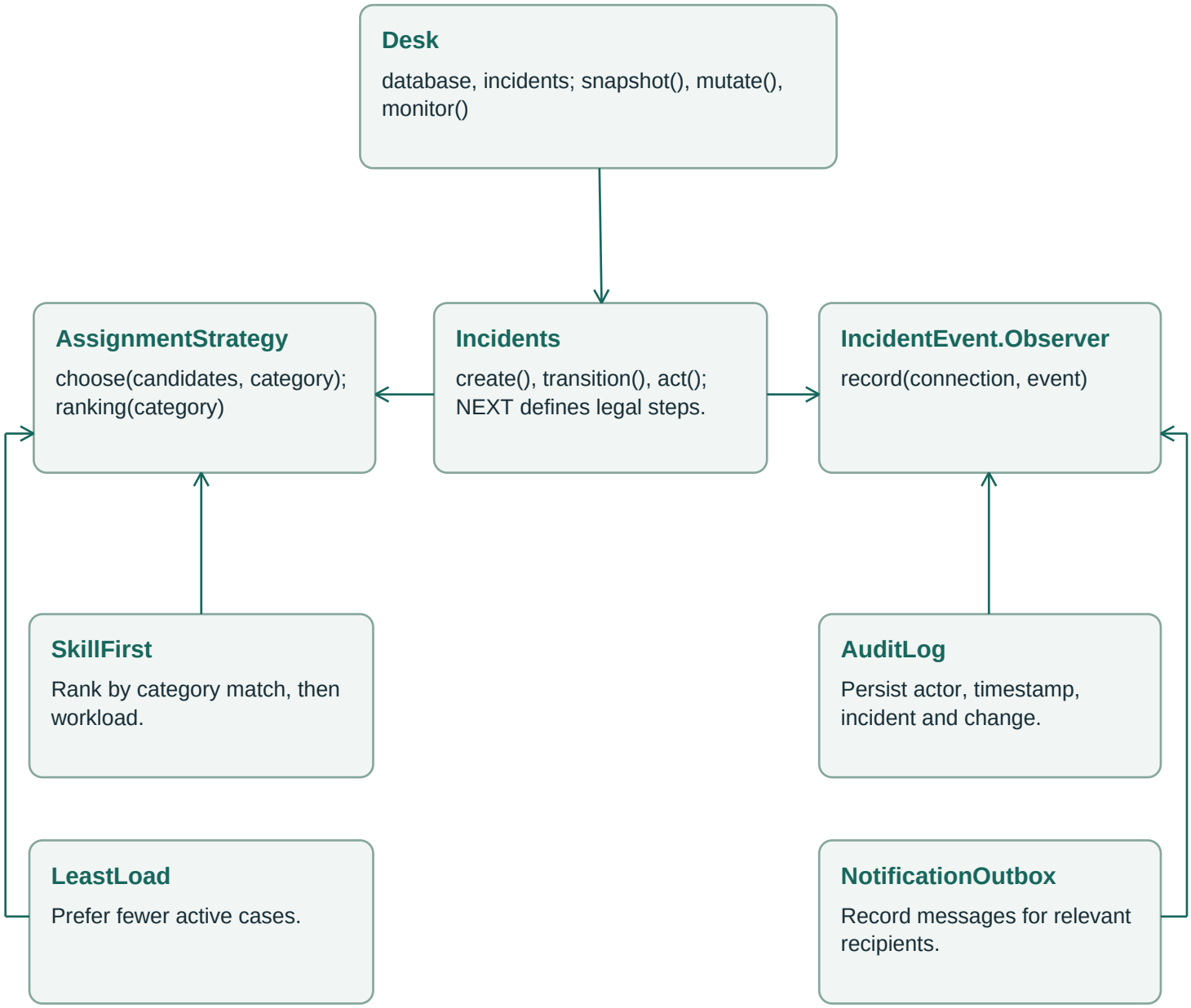


Trust boundary

Navigation exposes the relevant tasks for the current role. Each write and each read of private incident data also requires a server-side authorization decision. The API is the boundary that protects records.

Domain model

Class diagram / policy contracts and collaborating objects

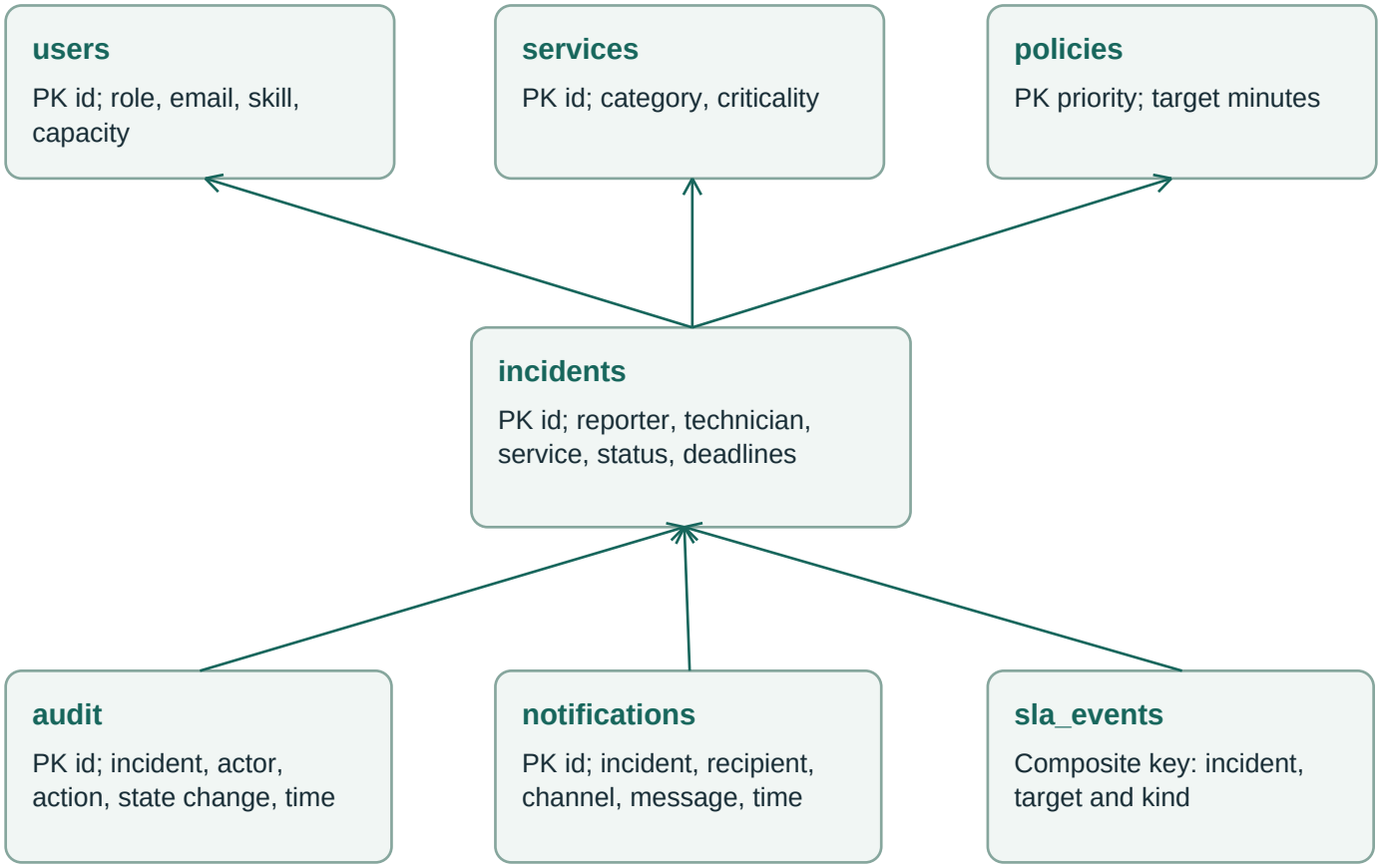


Composition and polymorphism

The domain service coordinates guards, assignment and event handling. Strategies implement the same selection contract. Incidents associate a reporter, affected service and assigned technician; each user or service can participate in many incidents.

Data design

Entity relationships / arrows point from child to parent



Additional relationships

Categories are a shared vocabulary for incidents and services. A related incident points to another incident. The settings table stores the selected assignment strategy. A notification belongs to one user; an incident can generate many messages and audit events.

Seed data

The local demonstration includes a payment gateway (criticality 3), an analytics database (2) and workplace devices (1). The seeded checkout timeout case shows a high-impact, high-urgency problem awaiting acknowledgement. Seed users include an employee, an administrator and technicians with different skills.

Schema and integrity

The executable schema in src/main/resources/schema.sql specifies every column, key and index. One transaction covers the incident change, audit entries and notification rows. A composite key prevents duplicate warnings for the same case, target and warning kind.

Decision rules and safeguards

Operational intelligence / security and limitations

Priority and routing

Multiply impact, urgency and service criticality, each scored 1 to 3. Products of 18 or above receive P1, 9 to 17 receive P2, 4 to 8 receive P3, and smaller products receive P4. Combining factors keeps isolated low-impact faults below broad failures. Skill-first ranks a category match ahead of workload; load-first ranks by active case count. Technician ID breaks ties.

SLA deadlines

The default response/resolution targets in minutes are P1 5/30, P2 15/120, P3 60/480 and P4 240/1440. Cases retain their original deadlines when policy settings change. Acknowledgement stops the response clock; resolution stops the resolution clock. The final fifth of an unfinished target is the warning window.

Related incidents

Within the same service, the system tokenizes titles and excludes common words. If the intersection of terms divided by their union reaches 0.30, it can suggest a related open case. Similar completed cases can supply a previous resolution. This transparent heuristic can miss synonyms and confuse similarly worded independent faults; technicians review its suggestions.

Access and storage

Passwords are salted and hashed. Random session tokens travel in HttpOnly, SameSite cookies. The server restricts incident reads and writes by role and ownership, checks request origins, bounds request sizes, and uses parameterized database statements. The local server binds to loopback by default.

Limits

The email channel is simulated. Monitoring requires a running application process. Timestamps reflect elapsed clock time; calendars for holidays and business hours would require a policy extension. The local demo credentials and development HTTP server are intended for evaluation.

Verification and demonstration

Executable test cases / run ./mvnw test

Authentication and visibility

1. Successful registration yields an employee account. 2. Duplicate registration fails. 3. Invalid email fails. 4. Short password fails. 5. Reopening the database preserves login. 6. Anonymous state request returns 401. 7. Incorrect password fails. 8. Logout invalidates the session token.

Authorization and transport

9. An employee cannot change routing. 10. A newly registered employee sees no cases owned by others. 11. Another reporter cannot add a note. 12. A cross-origin write is denied. 13. Array-shaped JSON is rejected. 14. An oversized payload is rejected. 15. Encoded parent-directory traversal cannot expose source.

Business rules

16. Priority boundary examples produce P1-P4. 17. Out-of-range impact rolls back creation. 18. A matching second report links the earlier case. 19. A case cannot move directly from Assigned to Closed. 20. The full technician/reporter lifecycle records timestamps, both notification channels, feedback and audit events. 21. Invalid service ID is rejected. 22. Least-load routing picks the least busy eligible technician. 23. Repeated SLA checks record a single breach per target.

Three demonstration workflows

Report an outage and inspect its priority, assignment and messages. Investigate as the assigned technician, resolve it, then verify and give feedback as the reporter. Report a related outage, switch routing policy and examine SLA warnings, workload and audit history. Detailed steps are in docs/demonstration.md.

Evaluation scope

The automated suite uses temporary SQLite databases and a real loopback HTTP listener. It exercises application behavior and authorization. A team presentation can supplement these checks with live role switching and usability observations.

Current implementation

Open and sign in

Run the Java 17 application from the repository root. GitHub stores the code; the Java process serves the local website.

```
./mvnw clean verify
```

```
java -jar target/relay.jar
```

Open http://127.0.0.1:8000/ while the server is running.

Demo accounts

Role	Username or email	Password
Administrator	admin@relay.test	RelayDemo2026!
Technician	tech@relay.test	RelayDemo2026!
Employee	employee@relay.test	RelayDemo2026!

Generated usernames

Every account also receives a system-generated username such as exampleA1B2C3D4@service.tech. The code differs by account and installation. Sign in with the demo email above, then open My profile using the button beside Sign out to see that account's exact username. These usernames do not receive email. A real contact email stays on the account; registered passwords are stored as hashes in SQLite.

Current feature additions

Accounts: personal profile editing, administrator account removal, generated role-based usernames, and technician assignment views in People.

Incidents: protected file attachments, reporter reopening, public and IT-only notes, P1 response escalation, and post-incident reviews with root cause, impact and prevention.

Knowledge and operations: searchable service articles, related-incident suggestions, routing strategies, SLA monitoring, notifications, audit history and reports.

For current step-by-step instructions and the complete feature list, read README.md, docs/features.md and docs/demonstration.md in the repository.